



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT ?

READ THE BOARD

WHAT THIS PROJECT BUILDS

Draw a tic-tac-toe board on paper, put the camera above it, and let the program read the board nine times a second. Red pieces become X, blue pieces become O, and an empty cell stays empty.

No arm movement. No rules. No winning. This project is the part that looks at the board, and it is deliberately the smallest interesting thing that can look at a board.

- **A region of interest** — one rectangle of the picture, defined by numbers, that everything else works inside.
- **A colour test per cell** — the red wraparound trick you already built in Project 5, reused on a smaller picture.
- **A board as data** — nine single characters in a 3x3 list, which is something a program can reason about.

● *The output of this project is not a game. It is a truth about the world, refreshed continuously: what is on the board, right now.*

Project 8 builds a real opponent on this `parse_board_state`. Everything you write this week feeds it.

NINE CELLS, ONE LOOP

The entire program is one loop that runs forever and does four things, over and over:

- 1 Read a frame**
One picture from the camera, 640 by 480.
- 2 Cut it into nine**
Nine rectangles, using arithmetic rather than a mouse.
- 3 Test each one**
Red, blue, or nothing. One character returned.
- 4 Draw it back**
The grid, the pieces, and a label, over the top of the picture.

Steps 2 and 3 are the whole project. Step 4 exists so that a human can check the answer without reading a single line of Python — and in this module that has turned out to matter more than expected.

- *Nothing here is slow. Nine small colour tests take well under a millisecond, which is why a plain loop with no threading is enough.*

BGR IS NOT A COLOUR NAME

OpenCV does not hand you "red". It hands you three numbers per pixel, in the order blue, green, red — and each one ranges from 0 to 255. So the brightest red is not (255, 0, 0), it is (0, 0, 255).

Testing a colour that way means writing down a range of possibilities for all three numbers at once. The conversion below moves the problem into a space where one number carries the hue:

```
hsv = cv2.cvtColor(cell_roi, cv2.COLOR_BGR2HSV)
```

- H is hue — where on the colour wheel the colour sits, 0 to 180 in OpenCV.
- S is saturation — how vivid it is. Low S means grey, whatever the hue says.
- V is value — how bright it is. Low V means dark, and dark red is black.

Converting does not make the threshold easier to write. It makes it possible to write one that survives a change in lighting, because hue is roughly stable under brightness changes and BGR is not.

HUE IS A CIRCLE

This is the detail that makes hue different from the other two channels, and it is the whole reason this project has two red ranges instead of one.

Hue starts at 0 for red and runs to 180 for red again. There is no 181. Red is at both ends of the scale, so a single `inRange` can never capture it.

```
RED = 0          ... and RED = 180      # the same colour, opposite ends
```

Two ranges are written to cover both ends, and each is a valid slice of the wheel on its own. The first is the 0 end, the second is the 180 end:

```
LOWER_RED1 = np.array([0, 120, 100])
UPPER_RED1 = np.array([10, 255, 255])
LOWER_RED2 = np.array([170, 120, 100])
UPPER_RED2 = np.array([180, 255, 255])
```

- *You wrote this exact structure in Project 5 for a red ball on a blue table. It is the same problem at a smaller scale, which is the point — a pattern worth learning once should be recognisable the second time.*

A MASK IS A PICTURE

Every colour test in OpenCV produces an image of the same size as its input, where each pixel is one of two values. Inside the range it is 255. Outside the range it is 0. That picture is called a mask, and it is why colour detection is arithmetic rather than guessing.

```
# Blue Mask  
mask_blue = cv2.inRange(hsv, LOWER_BLUE, UPPER_BLUE)
```

For blue, one range is enough, because blue sits in the middle of the hue scale and wraps around nothing:

```
LOWER_BLUE = np.array([90, 120, 100])  
UPPER_BLUE = np.array([130, 255, 255])
```

- *The mask for a red piece is built from two ranges, which means two masks. The next line combines them, and a pixel counts as red if it landed in either one.*

The S and V floors — 120 and 100 — stop a shadowed white cell from reading as a piece.

TWO MASKS, ONE COLOUR

Because red needed two ranges, it needs two masks — and then a way to treat them as a single answer:

```
mask_red1 = cv2.inRange(hsv, LOWER_RED1, UPPER_RED1)
mask_red2 = cv2.inRange(hsv, LOWER_RED2, UPPER_RED2)
mask_red = cv2.bitwise_or(mask_red1, mask_red2)
```

Read that third line as a sentence: a pixel is part of `mask_red` if it is part of `mask_red1` or part of `mask_red2`. The output is a new mask of the same size, not a count and not a boolean.

- *This is the line that would be wrong if the two ranges were merged with a single `inRange` covering 0 to 180. That would match every colour in between — orange, yellow, green, cyan, all of it — and the board would be full of pieces that are not there.*

Bitwise, not arithmetic. `bitwise_or` does not add pixel values together; it merges two pictures by rule. The distinction matters the moment a mask stops being a yes-or-no thing.

COUNTING PIXELS

You have two masks. Each one answers a question about every pixel, and the answer you actually want is a single number: how many of them said yes.

```
red_pixels = cv2.countNonZero(mask_red)
blue_pixels = cv2.countNonZero(mask_blue)
```

No thresholding step, no image comparison, no per-pixel loop in Python. OpenCV counts the non-zero entries and hands back an integer, which is exactly the number of pixels inside the colour range.

- **A number is easier to test than a picture.** Once you have 4,182 red pixels, every remaining decision is a comparison.
- **This runs nine times per frame** — once per cell — so the number has to be small. It is.

- *Counting is also why the cell size matters. The same 800-pixel threshold means something completely different in a 75x75 cell than in a 30x30 one, and that is the next idea.*

A THRESHOLD IS A FRACTION

The line that decides everything is this one:

```
MIN_PIXELS = 800
```

800 is not a guess: it is a fraction of the cell — here 75 by 75 pixels, 5,625 in total:

```
# cell area
75 * 75    # = 5625 px

# threshold as a fraction of that area
800 / 5625    # = 0.142, about 14%
```

So 800 means "a piece has to fill at least a seventh of its cell" — forgiving about shape, strict about colour.

- *Change the frame resolution and this number goes wrong silently — the geometry behind it must stay fixed.*

It is defined inside the function, so it is rebuilt on all nine calls per frame — cheap, but worth knowing.

CELL BY CELL

Everything above happens inside one function that is handed a small picture and returns one character. It knows nothing about grids, rows, columns, or the board:

```
def scan_cell(cell_roi):  
    """Inspects a single cell ROI and returns 'X', 'O', or ' '."""
```

One job, one return value, three possible answers. The caller will assemble nine of these characters into a board, but this function never has to know that.

```
    if red_pixels > MIN_PIXELS and red_pixels > blue_pixels:  
        return 'X'  
    elif blue_pixels > MIN_PIXELS and blue_pixels > red_pixels:  
        return 'O'  
    return ' '
```

Both halves matter: the threshold says the colour is present; the comparison says which one won.

- *A space means an untouched cell is already the answer.*

SLICING IS A RECTANGLE

Getting a single cell out of the frame is two brackets and four numbers. The frame is a list of rows, so the first bracket takes rows and the second takes columns inside each row:

```
frame[y1:y2, x1:x2]
```

The first bracket takes rows, the second takes columns. Row first, column second — that order is the only thing worth memorising here.

Order is row first, column second — the only part worth memorising. The result is a real array, not a view onto the original.

- *Nothing checks the rectangle is inside the picture.*

The calibration in this project keeps every cell inside the frame, so this is a thing to know rather than a bug to hit.

NINE CELLS TO A MATRIX

The outer function cuts nine rectangles and calls the first one nine times. The arithmetic is two lines because the grid is evenly divided:

```
for row in range(3):  
    for col in range(3):  
        x1 = GRID_X_MIN + col * CELL_W  
        y1 = GRID_Y_MIN + row * CELL_H  
        x2 = x1 + CELL_W  
        y2 = y1 + CELL_H
```

Multiplying by the cell size walks across or down; adding the grid origin converts a cell index into a pixel. Every number in the picture comes from that one pattern, which is why the drawing code later in the file looks almost

```
cell_roi = frame[y1:y2, x1:x2]  
board[row][col] = scan_cell(cell_roi)
```

Look closely at the indexing order: `board[row][col]` and `x1` from `col` while `y1` comes from `row`.

- *Swapping those two is the single most likely mistake in this function, and it produces a board that is transposed rather than wrong.*

ROW, COLUMN, INDEX

The board is a list of lists, and it starts as nine spaces — already correct before anything is detected:

```
board = [[' ' for _ in range(3)] for _ in range(3)]
```

Each `_` is a throwaway variable that repeats the inner pattern three times, giving three separate lists rather than three references to one. The same question comes up in Project 8.

```
board[0][0] = 'X'  
board[2][2] = 'O'
```

- *Printing the board is the fastest possible test of whether the indexing is right, and it costs one line. Put a token in the top left and check that it appears in the top left.*

This is the first project where the program holds a description of something in the world rather than a reaction to it. The list is the whole point of the project.

DRAWING THE ANSWER

A scanner nobody can check is not much use, so the loop writes its answer back onto the picture. Three calls:

```
cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 255, 255), 2)
```

Two corners and a thickness. Nine of these produce a grid, and you can check instantly whether the calibration is right — a grid that is off the paper is a grid that will misread the paper.

```
if token != ' ':  
    cv2.putText(frame, token, (x1 + CELL_W // 3, y1 + 2 * CELL_H // 3),  
                cv2.FONT_HERSHEY_SIMPLEX, 1.5, color, 3)
```

The character is written at roughly two thirds across and down, which is a compromise position that happens to look centred. The `token != ' '` test is what stops nine spaces from being drawn nine times over empty cells.

- *The colour is chosen one line earlier from the token itself, which is why the text is drawn in the same red or blue the detector just found.*

TELEMETRY IS EVIDENCE

One more line of text, placed in the corner, naming what the program thinks it is doing:

```
cv2.putText(frame, "Tic-Tac-Toe Vision Parser", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
```

If that line is missing, the window you are looking at is not from this program. That sounds trivial. It is not — a stale window from a previous run is the single most common way to spend an hour debugging a program that is working perfectly.

- **It proves the loop reached this line.** Not a guess — proof.
- **It gives the frame a version.** Useful when a behaviour changes between one run and the next.

This project prints far less than Project 6 did. There are no counters, no scores, no state. The evidence here is visual, and the overlay is the instrument panel.

- *Two lines in this file make claims about what the program does. One of them is true. You will find out which, and not from a stack trace.*

WHAT "SCANNER" DOES NOT MEAN

It is worth being precise about the scope, because the name suggests more than the file does. This program does not:

- **know the rules** — nothing checks for three in a row, and nothing decides whose turn it is.
- **move the arm** — the arm is powered on and sent home at startup, then never touched again.
- **remember anything** — the board from the previous frame is overwritten without ever being compared.
- **care about perspective** — the calibration assumes a flat board seen from roughly above.

● *Those are not omissions, they are the syllabus. Rules and robot moves are Project 8. A module that builds a whole game in one file builds something nobody can debug.*

CLAIMS AND EVIDENCE

Most of this module has taught you to write code that tells you when it fails. This project mostly works, and that creates a different problem: a working program gives you very little to check against.

So the habit for this week is the Project 6 habit applied to a project that mostly succeeds. Before you move on, find the claims, write them down, and verify each one by hand.

Two lines in this program make a statement about behaviour. One of them is checked every single time

- *you press a key, and you have probably pressed the key a hundred times without noticing that it does nothing.*
- **A promise made by a program** is a testable statement, exactly like a promise made by a person.
- **A promise that holds** is worth recording too. It is the only way to know which of your instincts are reliable.

Three of the four things this project appears to claim are true, and verifying those took as much work as catching the false one did.

BEFORE YOU TYPE ANYTHING

Copy the starter across and run it once before filling in a single TODO. This is the output, on a machine with no robot attached:

```
Robot ready.  
Traceback (most recent call last):  
  File "M2-P7-Starter.py", line 58, in <module>  
    LOWER_RED1 = np.array([0, 120, 100])  
                  ^^  
NameError: name 'np' is not defined
```

Two things to notice, and the first one is the interesting one.

- **It printed "Robot ready." first.** That line is outside the TODO you have not filled in yet, so it runs and reports success before anything has actually been initialised.
- **It failed on line 58, not line 131.** The first use of `np` is in a threshold constant, well before the camera code. Read errors from the top: the first missing name is the one to fix.

Nothing here is a bug in your starter file. This is what an empty TODO looks like, and seeing it once makes the shape of the problem obvious.

STEP 1 — IMPORT THE THREE LIBRARIES

The first TODO in the file. Three names you have used in every project so far:

```
import cv2
import numpy as np
import time
```

- `cv2` — the OpenCV functions, including everything colour related.
- `numpy as np` — the arrays that hold the colour thresholds. This project needs it where Project 6 did not.
- *If you skip this step, the first error you will see is the one from the previous slide, at line 58 — not at the line you would expect.*

This is the step that removes the `NameError` you just watched happen. Same error, same fix, and the error itself told you exactly which line was missing what.

STEP 2 — IMPORT THE ROBOT

One line. Importing it does not touch the hardware — it only makes the class available when the `try` block runs.

```
from pymycobot.mycobot280 import MyCobot280
```

Two package names, not one. `pymycobot` is the library; `mycobot280` is the module inside it; `MyCobot280` is the class you instantiate. The numbers in the name are the model number, not a version.

This import has to succeed even when you are running without an arm, because it happens at the top of the file before the `try` block gives you any protection at all. The library has to be installed; the hardware does not.

- *Remember this project: the arm is initialised and then never used again. The import is required by the file, not by the task.*

STEP 3 — CONNECT AND POWER ON

Inside the `try` block that already exists in the file.
Identical to Project 6:

```
print("Connecting to myCobot280...")

mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
mc.power_on()
time.sleep(0.5)
```

Two numbers that must match your hardware exactly: which serial port, and at what baud rate. The sleeps are there so the board can finish booting and acknowledge the power command before you send anything else.

If the arm is not connected, the constructor raises, the `except` block runs, and the program continues. We will come back to whether that promise is actually kept.

STEP 4 — MOVE TO THE HOME POSE

Three lines. The pose is different from Project 6, and the reason is given in the comment above it:

```
home_pos = [0, 45, -90, -45, 0, 0]
mc.send_angles(home_pos, 50)
time.sleep(2.0)
```

This is the folded position, defined by the module for this project so the arm is out of the way of the board the camera is looking at. The shape of the list is six joint angles, and 50 is a deliberately slow speed.

- *Compare with Project 6, where the pose lived at module level because two functions needed it. Here it is defined inside the `try` block and never used again — so its position in the file causes no problem at all. The rule is about use, not about where code sits.*

This is the last time you touch the arm in this entire project.

STEP 5 — CALCULATE THE GRID

The first TODO with real arithmetic in it. The constants above it are given to you, and they are the calibration:

```
GRID_SCALE = 0.75          # Scale factor for grid size (1.0 = original)
GRID_OFFSET_X = -10         # Horizontal offset in pixels (+X = right)
GRID_OFFSET_Y = -70         # Vertical offset in pixels (+Y = down)

_BASE_GRID_X_MIN, _BASE_GRID_X_MAX = 170, 470
_BASE_GRID_Y_MIN, _BASE_GRID_Y_MAX = 90, 390
```

Three numbers to change if your board is in the wrong place or the wrong size, and the TODO turns them into the four corners the rest of the program uses:

```
grid_w = (_BASE_GRID_X_MAX - _BASE_GRID_X_MIN) * GRID_SCALE
grid_h = (_BASE_GRID_Y_MAX - _BASE_GRID_Y_MIN) * GRID_SCALE
center_x = (_BASE_GRID_X_MIN + _BASE_GRID_X_MAX) / 2
center_y = (_BASE_GRID_Y_MIN + _BASE_GRID_Y_MAX) / 2

GRID_X_MIN = int(center_x - grid_w / 2 + GRID_OFFSET_X)
GRID_X_MAX = int(center_x + grid_w / 2 + GRID_OFFSET_X)
GRID_Y_MIN = int(center_y - grid_h / 2 + GRID_OFFSET_Y)
GRID_Y_MAX = int(center_y + grid_h / 2 + GRID_OFFSET_Y)
```

With the shipped numbers that is X 197 to 422, Y 57 to 282 — well inside a 640x480 frame.

STEP 6 — DIVIDE INTO THREE

A 3x3 board is just a region divided three times. Two lines:

```
CELL_W = (GRID_X_MAX - GRID_X_MIN) // 3  
CELL_H = (GRID_Y_MAX - GRID_Y_MIN) // 3
```

225 pixels wide and 225 tall, so each cell is exactly 75 by 75. That number is the one that matters later, because it is the denominator for the colour threshold.

- **The double slash** is floor division. It guarantees a whole number, which matters because the two results are subtracted from each other when the last cell is cut.
- **If the region is not evenly divisible**, floor division throws away the remainder rather than making one cell a pixel wider. A tiny error, and much better than nine mismatched cells.

Work out the value of 800 against 5,625 pixels now, before you need it later. It is a percentage, and knowing the percentage is what makes the threshold adjustable rather than magic.

STEP 7 — THE CELL FUNCTION

The signature and the docstring are inside this TODO, along with the conversion that turns a colour question into a hue question:

```
def scan_cell(cell_roi):  
    """Inspects a single cell ROI and returns 'X', 'O', or ' '."""  
    hsv = cv2.cvtColor(cell_roi, cv2.COLOR_BGR2HSV)
```

The docstring is not decoration. It states the return contract — three possible values — before a single line of logic exists, which is the cheapest way to make a function

This function will be called nine times per frame, so it is written to be completely ignorant of the board.

- *It receives a picture and returns a character. It has no idea nine of its answers will be assembled into rows.*

If you wanted to test this function without a camera, you would save one cell as a small image file and call `scan_cell` on it. Separating the question from the input is what makes that possible.

STEP 8 — THE RED MASK

Two ranges because red wraps around the hue scale, and a combination step because two ranges make two masks:

```
# Red Mask (handles HSV wraparound)
mask_red1 = cv2.inRange(hsv, LOWER_RED1, UPPER_RED1)
mask_red2 = cv2.inRange(hsv, LOWER_RED2, UPPER_RED2)
mask_red = cv2.bitwise_or(mask_red1, mask_red2)
```

Each `inRange` call produces a full-size black and white picture. The fourth line merges them into one mask that answers a single question: is this pixel red enough to count?

- *The comment is the documentation here, and it is load-bearing. Without the word "wraparound" the next reader sees a redundant-looking pair of masks and is likely to "simplify" them into one broken range.*

This is the third project in a row that uses this exact pattern. It is worth being able to write it without looking.

STEP 9 — THE BLUE MASK

Blue sits in the middle of the hue scale and wraps around nothing, so one range is genuinely enough:

```
# Blue Mask  
mask_blue = cv2.inRange(hsv, LOWER_BLUE, UPPER_BLUE)
```

The thresholds are given to you above the function. Read them as three questions: is the hue between 90 and 130, is the saturation at least 120, is the brightness at least 100.

The saturation floor is doing more work than the hue range. Pale blue and grey-blue pixels have a correct hue but a saturation below 120, and the floor is what rejects them.

- *Two ranges for one colour and one range for another is not inconsistent — it is the shape of the hue scale deciding how many slices the colour needs. Red sits at both ends, blue at one.*

STEP 10 — COUNT THE PIXELS

The simplest line in the project, and the one that turns a picture into something comparable:

```
red_pixels = cv2.countNonZero(mask_red)
blue_pixels = cv2.countNonZero(mask_blue)
```

Two integers. Everything downstream is a comparison between them and a constant, which means you could debug this entire project by printing numbers instead of looking at pictures.

- *Do that when it misbehaves. Print `red_pixels` and `blue_pixels` for one cell and you learn far more than from staring at the window. A count of 0 means the colour was never in range; a count of 200 means it was, but not enough.*

Also print `cell_roi.shape` if a cell seems to be reading wrong for no obvious reason. The shape tells you whether the region you cut was the size you expected.

STEP 11 — DECIDE THE TOKEN

The threshold is already defined above this TODO. Your job is the logic that uses it:

```
if red_pixels > MIN_PIXELS and red_pixels > blue_pixels:  
    return 'X'  
elif blue_pixels > MIN_PIXELS and blue_pixels > red_pixels:  
    return 'O'  
return ' '
```

- **First half of each test** — is this colour present at all? Below 800 pixels is noise, whatever colour it is.
- **Second half** — which colour dominates? A cell can easily hold more than 800 pixels of both.
- *The final return is not a fallback — it is the normal case. Nine out of ten cells on an empty board land here, and an empty cell should be the easiest thing this function can return.*

STEP 12 — THE BOARD FUNCTION

The signature is in this TODO. The rest of the function is already written for you above it:

```
def parse_board_state(frame):  
    """Scans all 9 grid cells and returns a 3x3 matrix."""
```

```
    for row in range(3):  
        for col in range(3):  
            x1 = GRID_X_MIN + col * CELL_W  
            y1 = GRID_Y_MIN + row * CELL_H  
            x2 = x1 + CELL_W  
            y2 = y1 + CELL_H
```

```
            cell_roi = frame[y1:y2, x1:x2]  
            board[row][col] = scan_cell(cell_roi)
```

All you typed is the function header. The behaviour was already decided; your job was to make it callable.

- *That is not a shortcut, it is the shape of the project: assemble logic that was already right.*

STEP 13 — OPEN THE CAMERA

Three lines, and the last two are new for this module:

```
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

A plain `VideoCapture`, not the threaded class from Project 6. Nothing in this loop blocks for long enough to need a thread, and a plain capture keeps the code short.

- *The 640x480 request is not a suggestion. `GRID_SCALE`, the offsets, and `MIN_PIXELS` are all calibrated against a 640x480 frame. OpenCV will often give you a different size anyway, and the calibration constants are what you adjust when it does.*

Print `frame.shape` once if the grid lands in the wrong place. The camera ignoring your requested resolution is a common and confusing source of that.

STEP 14 — OPEN THE LOOP

Two lines, and the `try` above them is already in the file:

```
try:
    while cap.isOpened() :
```

The loop tests whether the capture is still open, not whether there is a picture — that distinction matters at Step 15.

The `finally` block at the bottom is paired with this `try`: it releases the camera and closes the window whatever way you leave the loop.

```
finally:
    cap.release()
    cv2.destroyAllWindows()
```

- *Code under the `while` runs every frame, until you break out.*

Those are the only two lines in that block, and both are things that exist — note the total absence of the arm.

STEP 15 — READ A FRAME

The first thing that happens in every iteration:

```
ret, frame = cap.read()
if not ret:
    break
```

Two return values, and the first is the one people forget. `ret` is a boolean saying whether a picture was actually delivered; `frame` is the picture itself, or `None` if it failed.

- *The three lines are a set, not a sequence. Calling `read` without checking `ret` means the next line tries to slice a picture that does not exist. This is the difference between a camera that is unplugged mid-run and a program that stops with a sensible message.*

This is the same lesson as Project 5, where a dropped frame produced a confusing colour error instead of an obvious camera error.

STEP 16 — PARSE THE BOARD

One line, and it is the point of the whole project:

```
board_state = parse_board_state(frame)
```

Nine colour tests, returning a 3x3 list of characters. From this line onward, the program has an answer it can reason about rather than a picture it has to keep re-examining.

- **The name is different from the function.** `parse_board_state` is the function; `board_state` is this frame's result. The underscore is doing real work there.

- *You now have a truth about the board, nine times a second, in a form that could drive a robot. Everything after this line is either drawing it or showing it to a human.*

STEP 17 — DRAW THE GRID

You are inside the given double loop that walks the nine cells, one line below where `board_state` came from. Its own TODO is a single call:

```
cv2.rectangle(frame, (x1, y1), (x2, y2), (255, 255, 255), 2)
```

Look at where `x1`, `y1`, `x2` and `y2` come from. They are the given code in that loop, and they are the same four values `parse_board_state` used to cut the cells. One set of numbers, used twice, for cutting and for drawing.

- *That is why the grid appears exactly where the detection happened. If the white outline sits on your drawn lines, the calibration is right. If it does not, no amount of threshold tuning will fix anything.*

Drawing is a debugging tool before it is a user interface.

STEP 18 — DRAW THE TOKEN

The last multi-line TODO. Text, positioned inside the cell it belongs to:

```
if token != ' ':  
    cv2.putText(frame, token, (x1 + CELL_W // 3, y1 + 2 * CELL_H // 3),  
                cv2.FONT_HERSHEY_SIMPLEX, 1.5, color, 3)
```

Three arguments carry most of the meaning: the character to draw, where to draw it, and how big to draw it. The position is computed from `CELL_W` and `CELL_H`, so the text lands inside the right cell for any calibration.

The `if` test above it is what keeps nine empty spaces from being drawn nine times. Without it you get a board covered in faint grey spaces.

- *The character is drawn in red or blue because `color` was chosen from the detected token, not hard-coded. The picture is reporting its own finding back to you in colour.*

STEP 19 — WRITE THE TELEMETRY

One line, positioned in the corner of the frame rather than inside the grid:

```
cv2.putText(frame, "Tic-Tac-Toe Vision Parser", (20, 40), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 255, 255), 2)
```

Different from the other two `putText` calls in this file: smaller scale, thinner line, fixed coordinates, and no test in front of it. That last part is the design decision — it is unconditional, so it is on screen every single frame.

- *This line is also how you tell this program apart from any other OpenCV window you might have open. It names itself, once, every frame.*

STEP 20 — SHOW THE FRAME

The last TODO in the file:

```
cv2.imshow("Mission 02 - Project 07: Board State Scanner", frame)
```

Two arguments: a window title and the picture. The title is the one thing that identifies your program in the taskbar, and the string in it is the only place the project name appears on screen.

Notice this is not drawing — `imshow` puts the finished frame on screen. Everything before it modifies the frame in memory; this line is what you actually look at.

STEP 21 — READ THE KEYBOARD

The final TODO is the display line above. These two lines come with it, already written in the file:

```
if (cv2.waitKey(1) & 0xFF) == ord('q'):  
    break
```

Three things happening in one line. `waitKey(1)` waits about a millisecond and returns a key code. `& 0xFF` masks that code down to eight bits. `== ord('q')` compares it to the character `q`.

- **The wait is what makes the window responsive.** Without it the picture freezes and no key is ever read.
- **The mask is not optional.** Key codes can carry more than eight bits, and unmasked values will occasionally match 113 by accident.
- *Read that first line carefully, including the brackets. The exact parenthesisation is load-bearing, and there is a reason for it that we are about to look at.*

RUN IT

Send the file across and start it:

```
scp M2-P7-Starter.py er@192.168.1.149:Documents  
python M2-P7-Starter.py
```

Expected console output:

```
Connecting to myCobot280...  
Robot ready.  
Starting Tic-Tac-Toe Vision Scanner. Press 'q' to quit.
```

Then a window opens with your camera feed, a white grid, and the telemetry line in the top left. Place a red token in a cell and the letter X appears on it.

- *Three claims have now been made about this program. Two of them are about your setup; one is about the keyboard. All three are checkable by hand, and we are going to check them.*

VERIFY THE CLAIMS

Three claims. Checked one at a time, without changing any code.

#	The claim	Result
1	Runs without the robot attached	True
2	Identifies X and O correctly	True
3	Press q to quit	False

The first one deserves a note. It is the promise that was a lie in Project 6, so it was the obvious one to suspect — and it holds.

- *Verify every claim, not the ones you already distrust. The instinct to find a specific bug is not evidence, and a program that fails the test you expected is as surprising as one that passes.*

CLAIM 1 — THE FALLBACK HOLDS

Unplug the arm, run the program, and this is what you get:

```
Robot hardware warning: [Errno 2] No such file or directory: '/dev/ttyAMA0'  
Continuing with OpenCV camera pipeline only...  
  
Starting Tic-Tac-Toe Vision Scanner. Press 'q' to quit.
```

The program does keep going. Here is why, and the reason is structural rather than lucky:

- **mc is only used inside the try block.** Search the file after the handler and you will not find one.
- **The finally block does not touch it.** That block releases the camera and closes the window — two things that always exist.
- *Same message, same file shape, opposite outcome to Project 6. Identical wording tells you nothing; what the code does after the print is the whole answer.*

CLAIM 2 — FOUR THINGS LINED UP

The claim that the project's whole purpose rests on: red pieces read as X, blue pieces as O, empty cells as nothing.

Four properties had to be true at once for that claim to hold:

1. **The hue wraparound was handled.** Two ranges and a `bitwise_or`, so the 0 end and the 180 end of the scale are both reachable.
2. **The calibration put every cell inside the frame.** X 197 to 422 and Y 57 to 282 in a 640x480 picture.
3. **The threshold matched the cell size.** 800 of 5,625 pixels, about 14%.
4. **Row and column were not swapped.** `board[row][col]` fed by `y1` from `row` and `x1` from `col`.

Any one of those failing would produce a board that reads wrong in a way that looks like a colour problem. Four independent things had to be right, and all four were.

CLAIM 3 — THE KEY THAT DID NOTHING

The program announces its exit condition before it starts:

```
print("Starting Tic-Tac-Toe Vision Scanner. Press 'q' to quit.")
```

Press q during a run and the scanner keeps going. Press it twenty times. Nothing happens — no error, no message, no crash. The window has to be closed with the mouse or the program killed from another terminal.

This is the version from before this project was corrected:

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

The corrected file you have now reads `if (cv2.waitKey(1) & 0xFF) == ord('q'):` — identical apart from two brackets, which is exactly the point: the bug is the absence of something, and absence is easy to miss.

WHY TWO BRACKETS MATTER

Python does not read that line the way it looks. Comparisons bind tighter than bitwise operators, so the line groups as:

```
cv2.waitKey(1) & (0xFF == ord('q'))
```

The mask and the comparison swap places. Which makes the second half a constant, evaluated once and never again:

```
0xFF == 113    ->    False  
waitKey(1) & False    ->    0
```

And 0 is false, so the condition is never true and the `break` is unreachable. Every key, every frame, forever.

- *This is the most dangerous kind of bug: not a crash, not a wrong number, but a line that reads plausibly and evaluates to nothing at all.*

WHAT THAT ACTUALLY COST

Worth being concrete about the price of two missing brackets, because "the quit key did not work" undersells it.

- **The frame loop could not be left cleanly.** The only exit was killing the process, so the `finally` block never ran and the camera was not released.
- **Ctrl+C raised KeyboardInterrupt** partway through, which means the loop died wherever it happened to be rather than at a known point.
- **The camera stayed claimed by the process.** Running again quickly produced a camera that would not open — a second failure caused entirely by the first.
- **The banner taught a lie.** Anyone reading the printed instructions was told `q` worked. It did not, and nothing on screen suggested otherwise.

None of that is visible in the line itself. It appears only when you trace what the expression evaluates to.

THE HABIT

One bug, and it was found by the same method that found four in Project 6: write down what the program claims, then go and check.

- **Printed text is a promise.** A banner telling you how to exit is testable in ten seconds, and nobody tests it.
- **Check the claims you expect to pass.** Two of the three here were true, and confirming them is what made the third credible.
- **Brackets are not formatting.** Grouping in an expression is part of the meaning, and it is invisible at a glance.

This project was almost entirely correct. That is worth saying plainly, because the temptation after a quiet bug is to assume the file is full of them. It is not. One claim out of three was false, and it was the one nobody had any reason to doubt.

- *A program that mostly works is harder to check than one that crashes, because a crash tells you where to look.*

THE BRIEF

Read the Board

Nine squares. Each one is cut out of a camera frame, converted to a colour space where hue is meaningful, reduced to two pixel counts, and turned into a single character.

Do that nine times and you have the state of a game — which is the only thing a tic-tac-toe program actually needs to know. Not the picture. The state.

Which means this project is mostly arithmetic, and the arithmetic is the part that has to be right.

YOUR MISSION

Three parts, in order.

- 1. Recalibrate.** Change `GRID_SCALE` and the two offsets until the white grid lands on your drawn board. Record the three values you ended up with.
- 2. Move the threshold.** Set `MIN_PIXELS` low enough to read a piece that is only a corner of its cell, then work backwards to the highest value that still works. Record the number.
- 3. Break a claim on purpose.** Put both tokens in the same cell and record which one wins and why. Then take a piece away and record which count decides it.

All three are about the relationship between a number and what it means in the physical world — which is the actual subject of this project, and the reason it is not just an OpenCV exercise.



Somewhere in part 3 you will find yourself wanting to press q to get out. It works. That was not always true.

YOUR DATA LOG

Measurement	Your value	What you changed
GRID_SCALE		
GRID_OFFSET_X		
GRID_OFFSET_Y		
MIN_PIXELS		
Cell size (W x H)		

Write MIN_PIXELS as a percentage of the cell, not as a bare number.
The whole point of Step 11 was that 800 is really 14% — a percentage survives a change of resolution and a bare integer does not.

DEFINITION OF DONE

- ☐ All twenty-one TODOs filled in and the file runs on the Pi.
- ☐ No `TODO` markers left anywhere in the file.
- ☐ The white grid sits on your drawn board, in every cell.
- ☐ A red piece reads as X and a blue piece as O, from any position in the frame.
- ☐ An empty cell stays empty even under a shadow.
- ☐ `q` quits the program and the camera is released cleanly.

● *That last one is the difference between this file and the version it was corrected from. Both of them draw the board correctly.*

GO FURTHER

- **Break the calibration.** Set `GRID_OFFSET_Y` far enough off that cells fall outside the frame, and find out what a mis-sized ROI does to `cvtColor`.
- **Report the counts.** Print `red_pixels` and `blue_pixels` for the centre cell each frame, and watch them move as you slide a piece across it.
- **Notice the dead branch.** The `color` value for an empty cell is computed on every iteration and then never used. Find every one of those in your own code.
- **Compare consecutive frames.** Keep the last board and report only when it changes. That is the first step towards not re-reading a board that has not moved.

Pick one. Not all four.

DEBRIEF

This was the shortest project so far, and almost all of it was arithmetic.

The vision was the part we already knew — hue, masks, a threshold — and almost all of the real work was deciding where the grid was and how big each cell was. Two claims out of three were true, and the one that was false came down to a pair of missing brackets in a line that read perfectly well.

Nothing crashed. Nothing printed an error. The program simply did not do the one thing it said it would let you do. Write the claim down first. Run it second.